

Étude, modélisation et implémentation d'un bras robotique plan à deux degrés de liberté

avec simulation, trajectoires et perspective d'intégration GRBL

Projet de thèse

28 avril 2026

Table des matières

Résumé	4
1 Introduction générale	5
1.1 Contexte	5
1.2 Problématique	5
1.3 Objectifs du projet	5
1.4 Organisation du document	6
2 Présentation du système	7
2.1 Description mécanique	7
2.2 Repères et conventions	7
2.3 Hypothèses de modélisation	8
3 Fondements théoriques et modèles mathématiques	9
3.1 Cinématique directe	9
3.1.1 Principe	9
3.1.2 Équations	9
3.1.3 Interprétation	9
3.1.4 Implémentation	9
3.2 Cinématique inverse	10
3.2.1 Principe	10
3.2.2 Calcul de la distance à la cible	10
3.2.3 Condition d'atteignabilité	10
3.2.4 Calcul de l'angle du coude	10
3.2.5 Calcul de l'angle de base	11
3.2.6 Exemple numérique	11
3.2.7 Implémentation logicielle	11
3.3 Espace de travail	11
3.4 Jacobienne	12
3.4.1 Définition	12
3.4.2 Expression analytique	12
3.4.3 Rôle dans le projet	12
3.5 Singularités	12
3.5.1 Détection	12
3.5.2 Signification physique	13
3.5.3 Conséquence pratique	13

4	Génération des mouvements et stratégie de commande	14
4.1	Commande d'un point cible	14
4.2	Trajectoire linéaire dans l'espace cartésien	14
4.3	Trajectoires de démonstration	14
4.4	Interpolation articulaire pour l'animation	15
4.5	Nature de la commande	15
5	Implémentation logicielle du projet	16
5.1	Vue d'ensemble	16
5.2	Phase 1 : configuration et modélisation	16
5.2.1	Module <code>config.py</code>	16
5.2.2	Module <code>kinematics.py</code>	17
5.2.3	Module <code>simulator.py</code>	17
5.3	Phase 2 : préparation matérielle	17
5.3.1	Module <code>grbl_interface.py</code>	17
5.3.2	Module <code>motor_control.py</code>	18
5.3.3	Module <code>a4988_config.py</code>	18
5.4	Phase 3 : intégration complète	18
5.4.1	Module <code>robot_controller.py</code>	18
5.4.2	Module <code>gui.py</code>	18
5.5	Flux de données	19
5.6	Validation logicielle	19
6	Introduction à GRBL dans la phase prototype	20
6.1	Pourquoi GRBL ?	20
6.2	Principe d'intégration	20
6.3	Conversion angles vers pas	20
6.4	Abstraction vers les axes GRBL	21
6.5	Exemples de commandes G-code	21
6.6	Rôle du homing	22
6.7	Paramètres GRBL à considérer	22
6.8	Protocole expérimental recommandé pour le prototype	22
6.9	Limites actuelles et intérêt scientifique	23
7	Analyse des résultats à présenter	24
7.1	Résultats attendus en simulation	24
7.2	Captures d'écran recommandées	24
7.3	Lectures possibles des résultats	24
8	Perspective d'extension vers un troisième degré de liberté	25
8.1	Motivation	25
8.2	Deux scénarios possibles	25
8.3	Extension du modèle cinématique	25
8.4	Impact sur la cinématique inverse	26
8.5	Impact sur l'architecture logicielle	26
8.6	Impact sur le prototype	26
8.7	Intérêt scientifique de cette perspective	26
9	Conclusion générale	27

A	Extrait de code représentatif	28
B	Références bibliographiques	29

Résumé

Ce document présente de manière détaillée le projet de réalisation d'un bras robotique plan à deux degrés de liberté, commandé en position à partir d'un modèle cinématique et destiné à évoluer vers une validation expérimentale sur prototype physique. Le travail réunit trois dimensions complémentaires : la théorie mathématique, l'implémentation logicielle et la préparation à l'intégration matérielle.

Le coeur du projet repose sur la cinématique directe et inverse d'un manipulateur plan de type RR , c'est-à-dire deux articulations rotoïdes en série. Le modèle obtenu permet de passer d'une représentation articulaire (θ_1, θ_2) à une représentation cartésienne (x, y) , puis d'inverser cette relation pour atteindre un point cible. Sur cette base, le projet met en oeuvre plusieurs modes de fonctionnement : une simulation graphique, une exécution de trajectoires et un mode hybride combinant visualisation et dialogue avec une chaîne de commande inspirée de GRBL.

L'objectif final n'est pas seulement de déplacer un bras dans une animation, mais de montrer la cohérence entre les modèles mathématiques, les décisions logicielles et la future validation matérielle. Cette cohérence est essentielle dans un contexte de thèse, car elle permet de démontrer que l'implémentation numérique constitue déjà une preuve de faisabilité avant la fabrication complète du prototype.

Chapitre 1

Introduction générale

1.1 Contexte

Les bras robotiques à faible nombre de degrés de liberté constituent un excellent cadre d'étude pour aborder la robotique de manière rigoureuse. Ils permettent d'introduire les notions fondamentales de cinématique, d'espace de travail, de singularité, de génération de trajectoires et de commande, tout en conservant une complexité compatible avec une première validation expérimentale.

Dans ce projet, le système étudié est un bras robotique plan à deux degrés de liberté. Le bras comprend deux segments rigides, articulés autour de deux axes de rotation. Son effecteur final évolue dans le plan (x, y) , ce qui permet de relier naturellement les variables articulaires aux coordonnées cartésiennes d'un point à atteindre.

1.2 Problématique

La problématique générale peut être formulée ainsi :

Comment concevoir un système complet permettant de calculer les angles articulaires nécessaires pour atteindre une position donnée, de visualiser de manière crédible ce mouvement, puis de préparer le passage à une exécution réelle sur prototype commandé par GRBL ?

Cette problématique se décompose en quatre sous-questions :

1. Comment établir un modèle mathématique fiable du bras robotique ?
2. Comment transformer ce modèle en logiciel exécutable et observable ?
3. Comment structurer l'architecture pour préparer une liaison avec du matériel réel ?
4. Comment faire évoluer le système vers un troisième degré de liberté ?

1.3 Objectifs du projet

Les objectifs poursuivis sont les suivants :

- établir la cinématique directe et inverse d'un bras plan 2 DDL ;
- modéliser l'espace de travail et les singularités du robot ;
- implémenter une simulation graphique animée du mouvement ;

- générer et exécuter des trajectoires cartésiennes ;
- préparer une architecture logicielle compatible avec une chaîne matérielle pilotée par GRBL ;
- définir une perspective claire d’extension vers un robot à trois degrés de liberté.

1.4 Organisation du document

Le présent document suit une progression naturelle :

- le chapitre 2 présente le système étudié ;
- le chapitre 3 développe les modèles mathématiques ;
- le chapitre 4 décrit la génération des mouvements ;
- le chapitre 5 détaille l’implémentation logicielle ;
- le chapitre 6 introduit l’intégration future de GRBL sur prototype ;
- le chapitre 8 expose l’extension vers un troisième degré de liberté.

Chapitre 2

Présentation du système

2.1 Description mécanique

Le robot considéré est un manipulateur plan à deux articulations rotoïdes. Il est formé :

- d'une base fixe ;
- d'un premier segment de longueur L_1 ;
- d'un second segment de longueur L_2 ;
- d'un effecteur final situé à l'extrémité du second segment.

Dans l'état actuel du projet, les paramètres utilisés dans le logiciel sont :

TABLE 2.1 – Paramètres principaux du bras robotique

Paramètre	Symbole	Valeur
Longueur du premier segment	L_1	200 mm
Longueur du second segment	L_2	150 mm
Limite articulaire 1	θ_1	$[-\pi, \pi]$
Limite articulaire 2	θ_2	$[-150^\circ, 150^\circ]$
Pas moteur par révolution	–	200
Micro-pas	–	16
Fréquence d'animation	–	60 fps

2.2 Repères et conventions

Le repère de travail est choisi de la manière suivante :

- l'origine est placée à la base du robot ;
- l'axe x est horizontal et orienté vers la droite ;
- l'axe y est vertical et orienté vers le haut ;
- les angles sont comptés dans le sens trigonométrique.

L'articulation 1 correspond à la rotation de base, tandis que l'articulation 2 correspond à la rotation du coude.

Emplacement réservé pour une capture d'écran

Insérer ici un schéma ou une capture représentant la base, les deux segments, les angles θ_1 et θ_2 , ainsi que l'effecteur final.

FIGURE 2.1 – Schéma cinématique du bras robotique 2 DDL

2.3 Hypothèses de modélisation

Pour conserver un cadre analytique clair, les hypothèses suivantes sont adoptées :

- les segments sont supposés rigides ;
- les liaisons sont parfaites et sans jeu dans la modélisation théorique ;
- le mouvement est purement plan ;
- les effets dynamiques, les frottements et les couples ne sont pas encore intégrés dans la loi de commande principale ;
- le contrôle est actuellement de type position, en boucle ouverte, avec validation d'abord par simulation.

Ces hypothèses sont adaptées à une première phase de validation, où l'objectif principal est la cohérence cinématique et la faisabilité expérimentale.

Chapitre 3

Fondements théoriques et modèles mathématiques

3.1 Cinématique directe

3.1.1 Principe

La cinématique directe consiste à calculer la position cartésienne de l'effecteur final à partir des angles articulaires. Pour un bras plan 2 DDL, cette relation est obtenue par composition vectorielle des deux segments.

3.1.2 Équations

La position du premier joint intermédiaire est :

$$x_1 = L_1 \cos(\theta_1), \quad (3.1)$$

$$y_1 = L_1 \sin(\theta_1). \quad (3.2)$$

La position de l'effecteur final devient alors :

$$x = L_1 \cos(\theta_1) + L_2 \cos(\theta_1 + \theta_2), \quad (3.3)$$

$$y = L_1 \sin(\theta_1) + L_2 \sin(\theta_1 + \theta_2). \quad (3.4)$$

Ces expressions constituent le modèle central de la cinématique directe.

3.1.3 Interprétation

La première contribution correspond à la projection du premier bras sur les axes du repère. La seconde contribution correspond à la projection du second bras, dont l'orientation absolue est la somme des deux angles articulaires.

3.1.4 Implémentation

Dans l'application, cette partie est prise en charge par le module `phase1_simulation/kinematics.py` au travers de la méthode `forward_kinematics`. Ce calcul est utilisé :

- pour afficher la position courante ;
- pour dessiner le bras dans le simulateur ;
- pour tracer l'historique de trajectoire ;
- pour vérifier la cohérence de la cinématique inverse.

3.2 Cinématique inverse

3.2.1 Principe

La cinématique inverse consiste à déterminer les angles (θ_1, θ_2) permettant d'atteindre une cible cartésienne (x, y) . Cette opération est plus délicate que la cinématique directe, car elle peut conduire à plusieurs solutions ou à aucune solution.

3.2.2 Calcul de la distance à la cible

La distance entre la base et le point cible vaut :

$$r = \sqrt{x^2 + y^2}. \quad (3.5)$$

Cette grandeur permet de savoir immédiatement si la cible appartient à l'espace de travail.

3.2.3 Condition d'atteignabilité

Le point est atteignable si et seulement si :

$$|L_1 - L_2| \leq r \leq L_1 + L_2. \quad (3.6)$$

Pour les valeurs actuelles du projet, on obtient :

$$R_{\max} = L_1 + L_2 = 350 \text{ mm}, \quad (3.7)$$

$$R_{\min} = |L_1 - L_2| = 50 \text{ mm}. \quad (3.8)$$

3.2.4 Calcul de l'angle du coude

L'application de la loi des cosinus fournit :

$$\cos(\theta_2) = \frac{r^2 - L_1^2 - L_2^2}{2L_1L_2}. \quad (3.9)$$

On en déduit :

$$\theta_2 = \pm \arccos \left(\frac{r^2 - L_1^2 - L_2^2}{2L_1L_2} \right). \quad (3.10)$$

Les deux signes correspondent aux deux configurations classiques :

- coude vers le haut ;
- coude vers le bas.

3.2.5 Calcul de l'angle de base

Une fois θ_2 déterminé, l'angle de base s'écrit :

$$\theta_1 = \text{atan2}(y, x) - \text{atan2}(L_2 \sin(\theta_2), L_1 + L_2 \cos(\theta_2)). \quad (3.11)$$

3.2.6 Exemple numérique

Considérons la cible $(x, y) = (250, 150)$ mm avec $L_1 = 200$ mm et $L_2 = 150$ mm.

$$r = \sqrt{250^2 + 150^2} = 291.55 \text{ mm}, \quad (3.12)$$

$$\cos(\theta_2) = \frac{291.55^2 - 200^2 - 150^2}{2 \times 200 \times 150} \approx 0.374. \quad (3.13)$$

On en déduit une solution de type coude vers le haut de l'ordre de :

$$\theta_2 \approx 68.0^\circ, \quad (3.14)$$

$$\theta_1 \approx 2.5^\circ. \quad (3.15)$$

Cette solution est cohérente avec l'implémentation actuellement validée dans le code.

3.2.7 Implémentation logicielle

L'algorithme de cinématique inverse est encapsulé dans la méthode `inverse_kinematics`. Le logiciel :

- vérifie d'abord l'atteignabilité du point ;
- calcule ensuite $\cos(\theta_2)$;
- borne numériquement cette valeur dans l'intervalle $[-1, 1]$;
- choisit l'une des deux branches ;
- vérifie enfin les limites articulaires.

3.3 Espace de travail

L'espace de travail du bras est une couronne circulaire comprise entre les rayons $R_{\min}$ et $R_{\max}$.

Cette géométrie a une conséquence importante pour l'interface utilisateur : un point visuellement situé trop près de la base ne doit pas être considéré comme atteignable, tout comme un point situé à l'extérieur du rayon maximum.

L'affichage graphique du projet représente explicitement cette couronne afin que l'utilisateur puisse interpréter visuellement les zones accessibles et inaccessibles.

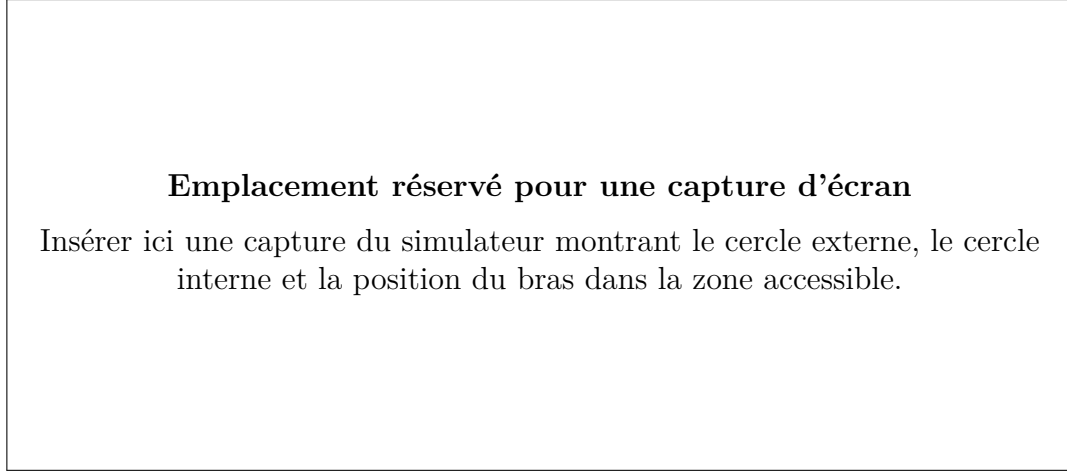


FIGURE 3.1 – Visualisation de l’espace de travail

3.4 Jacobienne

3.4.1 Définition

La matrice jacobienne relie les vitesses articulaires aux vitesses cartésiennes :

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = J(\theta_1, \theta_2) \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix}. \quad (3.16)$$

3.4.2 Expression analytique

En dérivant les équations (3.3) et (3.4), on obtient :

$$J(\theta_1, \theta_2) = \begin{bmatrix} -L_1 \sin(\theta_1) - L_2 \sin(\theta_1 + \theta_2) & -L_2 \sin(\theta_1 + \theta_2) \\ L_1 \cos(\theta_1) + L_2 \cos(\theta_1 + \theta_2) & L_2 \cos(\theta_1 + \theta_2) \end{bmatrix}. \quad (3.17)$$

3.4.3 Rôle dans le projet

Même si le projet n’utilise pas encore la jacobienne comme outil principal de commande, sa présence est importante pour trois raisons :

- analyser les singularités ;
- préparer une évolution vers une commande en vitesse ou en effort ;
- fournir un socle théorique solide pour la suite de la thèse.

3.5 Singularités

3.5.1 Détection

Une singularité apparaît lorsque le déterminant de la jacobienne est nul :

$$\det(J) = L_1 L_2 \sin(\theta_2) = 0. \quad (3.18)$$

Donc les configurations singulières apparaissent lorsque :

$$\theta_2 = 0 \quad \text{ou} \quad \theta_2 = \pi. \quad (3.19)$$

3.5.2 Signification physique

Ces cas correspondent à des configurations alignées, soit complètement déployées, soit complètement repliées. Dans ces situations, certaines directions de mouvement cartésien deviennent difficiles, voire impossibles, à produire localement.

3.5.3 Conséquence pratique

La prise en compte des singularités est importante pour la planification de trajectoire, car une trajectoire mathématiquement valide peut devenir défavorable en pratique si elle s'approche trop d'une zone où les vitesses articulaires devraient devenir très élevées.

Chapitre 4

Génération des mouvements et stratégie de commande

4.1 Commande d'un point cible

Le cas le plus simple consiste à atteindre un point cible (x_c, y_c) . La chaîne de traitement est alors :

1. saisie ou sélection de la cible ;
2. calcul de la cinématique inverse ;
3. validation des limites ;
4. mise à jour des consignes articulaires ;
5. interpolation du mouvement pour l'animation ;
6. exécution simulée, matérielle ou hybride.

4.2 Trajectoire linéaire dans l'espace cartésien

Pour relier un point initial (x_0, y_0) à un point final (x_f, y_f) , le projet met en oeuvre une interpolation cartésienne discrète :

$$x_i = x_0 + \frac{i}{N-1}(x_f - x_0), \quad (4.1)$$

$$y_i = y_0 + \frac{i}{N-1}(y_f - y_0), \quad (4.2)$$

où N est le nombre de points discrétisés.

Pour chaque point intermédiaire, la cinématique inverse est recalculée. Cette stratégie présente deux intérêts :

- le chemin suivi dans le plan est conforme à l'intuition visuelle ;
- la simulation devient plus démonstrative qu'un saut direct vers la position finale.

4.3 Trajectoires de démonstration

Au-delà du déplacement vers un point, plusieurs trajectoires de démonstration ont été structurées dans le projet :

- trajectoire linéaire ;
- trajectoire carrée ;
- trajectoire circulaire ;
- trajectoire en zigzag.

Ces trajectoires servent un double objectif :

- valider la robustesse du calcul cinématique sur plusieurs points successifs ;
- fournir des résultats visuels convaincants pour une démonstration ou une soutenance.

4.4 Interpolation articulaire pour l'animation

Pour qu'un mouvement soit observable et crédible, le projet n'effectue pas une simple mise à jour instantanée des angles. Une interpolation temporelle est introduite entre l'état initial et l'état final.

Si θ_k^0 et θ_k^f désignent les angles initial et final d'une articulation $k \in \{1, 2\}$, alors l'évolution peut être écrite sous la forme :

$$\theta_k(t) = \theta_k^0 + s(t) (\theta_k^f - \theta_k^0), \quad (4.3)$$

où $s(t)$ est une fonction de lissage vérifiant $s(0) = 0$ et $s(T) = 1$.

Dans l'implémentation actuelle, une loi de type cosinus est utilisée :

$$s(\alpha) = \frac{1}{2} - \frac{1}{2} \cos(\pi\alpha), \quad \alpha \in [0, 1]. \quad (4.4)$$

Cette écriture évite un déplacement visuellement brutal au démarrage et à l'arrivée.

4.5 Nature de la commande

Le projet relève d'une commande en position, principalement en boucle ouverte. Autrement dit, la consigne est calculée à partir du modèle, puis transmise au système de simulation ou au futur système matériel sans mesure continue de correction dans la boucle principale.

Ce choix est adapté à une phase de prototypage académique, car il permet :

- de valider la chaîne de calcul ;
- d'identifier les erreurs de modélisation ;
- de préparer une future extension vers une commande plus élaborée si nécessaire.

Chapitre 5

Implémentation logicielle du projet

5.1 Vue d'ensemble

Le dépôt est organisé en trois niveaux fonctionnels :

1. simulation et modélisation ;
2. commande matérielle ;
3. intégration complète.

Listing 5.1 – Organisation générale du projet

```
phase1_simulation/  
    config.py  
    kinematics.py  
    simulator.py  
  
phase2_hardware/  
    grbl_interface.py  
    motor_control.py  
    a4988_config.py  
  
phase3_integration/  
    robot_controller.py  
    gui.py  
  
tests/  
    test_kinematics.py  
    test_robot_controller.py
```

5.2 Phase 1 : configuration et modélisation

5.2.1 Module `config.py`

Ce module centralise les paramètres géométriques, angulaires, moteurs et graphiques. Il sert de point unique de cohérence pour l'ensemble du projet. Cette approche évite la dispersion des constantes dans le code et facilite les recalibrages.

5.2.2 Module `kinematics.py`

Ce module implémente :

- la cinématique directe ;
- la cinématique inverse ;
- la jacobienne ;
- le calcul de la frontière de l'espace de travail ;
- la génération de trajectoires linéaires ;
- la génération de trajectoires polygones et circulaires.

Il constitue donc le noyau mathématique du projet.

5.2.3 Module `simulator.py`

Le simulateur graphique en `Pygame` représente :

- la base du robot ;
- les deux segments ;
- l'effecteur final ;
- la cible ;
- l'historique de trajectoire ;
- l'espace de travail sous forme de couronne.

L'évolution récente du projet a consisté à rendre cette simulation véritablement animée. Le bras n'est plus figé ni téléporté : les angles sont interpolés sur une durée finie, ce qui améliore la lisibilité et la valeur démonstrative du système.

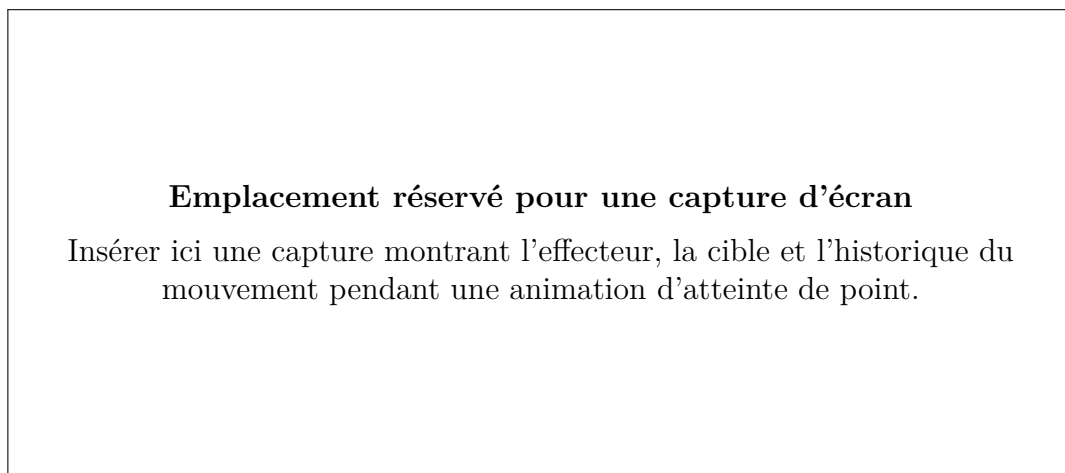


FIGURE 5.1 – Simulation graphique du mouvement vers un point cible

5.3 Phase 2 : préparation matérielle

5.3.1 Module `grbl_interface.py`

Ce module encapsule la communication série avec GRBL. Il gère notamment :

- l’ouverture et la fermeture du port série ;
- l’envoi de commandes G-code ;
- la lecture des réponses de type `ok` ou `error:N` ;
- la lecture du statut de la carte ;
- les commandes de homing, de reset et de déverrouillage.

5.3.2 Module `motor_control.py`

Ce module constitue une couche d’abstraction entre les angles articulaires et les commandes envoyées à GRBL. Son rôle est important, car il matérialise la transition entre le monde mathématique et le monde électromécanique.

5.3.3 Module `a4988_config.py`

Ce module regroupe les éléments de configuration liés au driver A4988 :

- modes de micro-pas ;
- réglage du courant ;
- rappels de câblage ;
- paramètres utiles pour la préparation du prototype.

5.4 Phase 3 : intégration complète

5.4.1 Module `robot_controller.py`

Le contrôleur principal fédère les trois modes du projet :

- **simulation** : calcul et animation sans matériel ;
- **hardware** : envoi des commandes vers GRBL ;
- **hybrid** : animation et exécution matérielle simultanées.

Ce module gère également :

- l’état courant du robot ;
- l’historique des positions ;
- les trajectoires planifiées ;
- les rappels d’état destinés à l’interface graphique ;
- l’animation temporelle des mouvements.

5.4.2 Module `gui.py`

L’interface `Tkinter` permet de piloter le robot à un niveau plus démonstratif. Elle intègre :

- un contrôle par angles ;
- un contrôle par position cartésienne ;
- une planification de trajectoire ;

- des trajectoires prédéfinies ;
- une console de messages ;
- une vue temps réel du bras.

L'intérêt scientifique de cette interface est majeur : elle ne sert pas seulement à piloter, mais aussi à rendre visible la correspondance entre la théorie, le code et le comportement du robot.

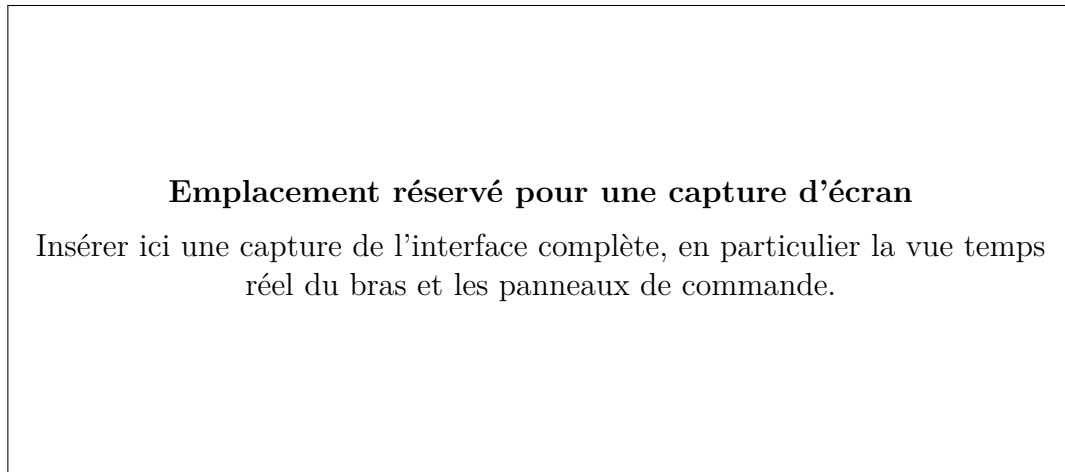


FIGURE 5.2 – Interface graphique intégrée du projet

5.5 Flux de données

Le flux de données peut être résumé ainsi :

1. l'utilisateur demande un mouvement ;
2. le contrôleur interprète la demande ;
3. la cinématique inverse calcule les angles requis ;
4. le contrôleur anime l'évolution articulaire ;
5. en mode matériel ou hybride, une couche convertit la consigne en dialogue GRBL ;
6. l'interface restitue l'état courant et la trajectoire.

5.6 Validation logicielle

Le projet contient des tests unitaires dédiés aux modules critiques, notamment :

- tests de cinématique directe ;
- tests de cinématique inverse ;
- tests de cohérence angles/pas ;
- tests sur la planification et l'exécution de trajectoires.

Ces tests jouent un rôle fondamental : ils garantissent que la démonstration visuelle repose sur un comportement mathématique vérifié, et non sur une simple animation.

Chapitre 6

Introduction à GRBL dans la phase prototype

6.1 Pourquoi GRBL ?

GRBL est un micrologiciel libre largement utilisé pour piloter des moteurs pas à pas à partir de commandes G-code. Il est historiquement conçu pour les machines à commande numérique, mais son principe de fonctionnement est suffisamment générique pour être adapté à un bras robotique de petite taille, à condition de définir une couche d'interprétation appropriée.

Dans ce projet, GRBL est envisagé comme une solution pragmatique pour le prototype, car il fournit déjà :

- la communication série ;
- la gestion des vitesses d'avance ;
- la génération de pas ;
- les mécanismes de homing et de sécurité ;
- un protocole robuste et documenté.

6.2 Principe d'intégration

L'idée n'est pas de faire croire à GRBL qu'il pilote directement un bras dans l'espace cartésien du plan, mais de l'utiliser comme couche de bas niveau pour produire les déplacements des moteurs correspondant aux angles articulaires.

Le principe retenu est le suivant :

1. le modèle cinématique calcule (θ_1, θ_2) ;
2. ces angles sont convertis en nombres de pas moteurs ;
3. une relation de calibration convertit ces pas en positions compatibles avec les axes GRBL ;
4. GRBL reçoit alors une commande sur ses axes X et Y.

6.3 Conversion angles vers pas

Soit N_1 et N_2 les nombres de pas équivalents aux angles articulaires :

$$N_1 = \theta_1 S_1, \quad (6.1)$$

$$N_2 = \theta_2 S_2, \quad (6.2)$$

où :

$$S_1 = \frac{N_{\text{rev}} \mu_1 g_1}{2\pi}, \quad (6.3)$$

$$S_2 = \frac{N_{\text{rev}} \mu_2 g_2}{2\pi}. \quad (6.4)$$

Ici :

- N_{rev} est le nombre de pas par révolution du moteur ;
- μ_i est le facteur de micro-pas ;
- g_i est le rapport de réduction mécanique éventuel.

Dans l'état logiciel actuel, cette conversion est déjà présente via `steps_per_rad_1` et `steps_per_rad_2`.

6.4 Abstraction vers les axes GRBL

Une couche de mapping associe :

- l'axe X de GRBL à l'articulation 1 ;
- l'axe Y de GRBL à l'articulation 2.

Dans une première version, on peut poser :

$$X_{\text{grbl}} = k_s N_1, \quad (6.5)$$

$$Y_{\text{grbl}} = k_s N_2, \quad (6.6)$$

où k_s est un coefficient de calibration exprimé en millimètres GRBL par pas moteur.

Il est important de noter que ce choix relève d'une abstraction d'intégration et non d'une vérité géométrique universelle. Lors du prototype physique, cette relation devra être calibrée expérimentalement afin que la consigne envoyée corresponde exactement au déplacement angulaire réel.

6.5 Exemples de commandes G-code

Une commande de déplacement absolu peut prendre la forme :

Listing 6.1 – Exemple de commande G-code pour une consigne articulaire

```
G90 G1 X12.500 Y8.200 F1000
```

où :

- `G90` active le mode absolu ;

- G1 demande une interpolation linéaire ;
- X et Y représentent ici les deux articulations abstraites ;
- F impose la vitesse d'avance.

D'autres commandes importantes sont :

Listing 6.2 – Commandes utiles pour la phase prototype

\$H	% homing
\$X	% unlock apres alarme
?	% lecture du statut

6.6 Rôle du homing

Sur le prototype, le homing sera indispensable pour référencer la machine. Sans cette opération, aucune cohérence durable entre la simulation et le robot réel ne peut être garantie.

Le homing permettra :

- de retrouver une origine articulaire connue ;
- de limiter la dérive cumulative des positions ;
- de sécuriser les essais ;
- de comparer les expériences dans des conditions reproductibles.

6.7 Paramètres GRBL à considérer

Les paramètres les plus sensibles dans la phase prototype seront :

- les pas par millimètre abstrait des axes \$100 et \$101 ;
- les vitesses maximales \$110 et \$111 ;
- les accélérations \$120 et \$121 ;
- l'activation du homing ;
- les limites logicielles et matérielles.

6.8 Protocole expérimental recommandé pour le prototype

Lorsque le prototype sera disponible, une séquence de validation progressive est recommandée :

1. vérifier le câblage et le réglage des courants des A4988 ;
2. établir la communication série avec GRBL ;
3. effectuer un homing fiable ;
4. commander de faibles mouvements articulaires ;
5. mesurer la correspondance entre consigne et déplacement réel ;

6. calibrer les coefficients de conversion ;
7. tester l'atteinte d'un point ;
8. tester ensuite une trajectoire complète en mode hybride.

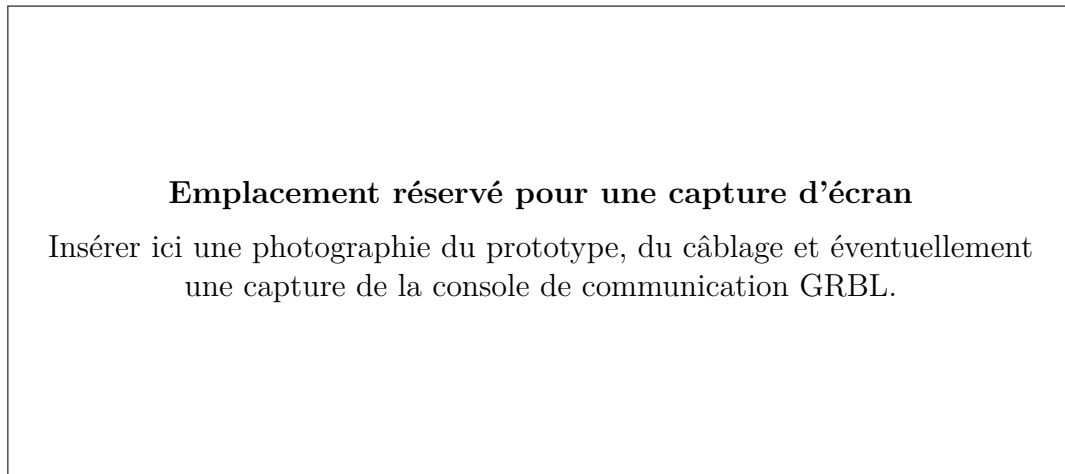


FIGURE 6.1 – Prototype physique et intégration GRBL

6.9 Limites actuelles et intérêt scientifique

À ce stade, l'intégration GRBL doit être comprise comme une passerelle vers le prototype. Le système démontre déjà la logique complète : calcul d'angles, animation, préparation des trajectoires, abstraction vers des commandes matérielles. Cette approche est scientifiquement pertinente, car elle isole clairement les niveaux de complexité :

- la géométrie du problème ;
- la logique de commande ;
- l'exécution électromécanique.

Chapitre 7

Analyse des résultats à présenter

7.1 Résultats attendus en simulation

La validation visuelle doit montrer que :

- le bras suit réellement la cible au lieu de rester figé ;
- la trajectoire affichée correspond au chemin voulu ;
- la distinction entre mode point, mode trajectoire et mode hybride est visible ;
- l'historique de mouvement permet d'interpréter la cohérence de l'exécution.

7.2 Captures d'écran recommandées

Pour documenter efficacement le travail, il sera utile d'insérer :

1. une capture du simulateur au repos avec l'espace de travail visible ;
2. une capture d'une atteinte de point ;
3. une capture d'une trajectoire carrée ou circulaire ;
4. une capture de l'interface graphique complète ;
5. une capture du mode hybride lors de l'échange avec GRBL ;
6. une photographie du prototype lorsqu'il sera disponible.

7.3 Lectures possibles des résultats

Dans l'analyse, il conviendra d'insister sur le fait que la simulation n'est pas une simple illustration décorative. Elle constitue un outil de validation scientifique, car elle permet de vérifier :

- la cohérence des équations ;
- l'impact des limites articulaires ;
- la forme de l'espace de travail ;
- le comportement en voisinage de trajectoires plus complexes.

Chapitre 8

Perspective d'extension vers un troisième degré de liberté

8.1 Motivation

L'ajout d'un troisième degré de liberté constitue la suite naturelle du projet. Un bras 2 DDL permet de positionner l'effecteur dans un plan, mais ne contrôle pas son orientation de manière indépendante. Or, dans un grand nombre d'applications robotiques, la position seule ne suffit pas : il faut également maîtriser l'orientation de l'outil.

8.2 Deux scénarios possibles

Deux évolutions sont particulièrement naturelles :

- l'ajout d'une troisième articulation rotoïde pour contrôler l'orientation terminale ;
- l'ajout d'un axe prismatique ou vertical si l'application exige une variation de hauteur.

Dans la continuité la plus directe du projet actuel, le scénario le plus cohérent est l'ajout d'une troisième articulation rotoïde, ce qui conduit à un bras plan de type *RRR*.

8.3 Extension du modèle cinématique

Avec une troisième articulation de longueur L_3 et d'angle θ_3 , la position de l'effecteur devient :

$$x = L_1 \cos(\theta_1) + L_2 \cos(\theta_1 + \theta_2) + L_3 \cos(\theta_1 + \theta_2 + \theta_3), \quad (8.1)$$

$$y = L_1 \sin(\theta_1) + L_2 \sin(\theta_1 + \theta_2) + L_3 \sin(\theta_1 + \theta_2 + \theta_3). \quad (8.2)$$

Si l'on souhaite en plus imposer une orientation terminale ϕ , on obtient :

$$\phi = \theta_1 + \theta_2 + \theta_3. \quad (8.3)$$

Le problème n'est alors plus seulement de positionner l'effecteur, mais aussi de fixer son attitude finale.

8.4 Impact sur la cinématique inverse

Le passage au cas *RRR* augmente nettement la richesse du problème :

- il devient possible d'imposer simultanément position et orientation ;
- le nombre de solutions peut augmenter ;
- la gestion des limites articulaires devient plus délicate ;
- la jacobienne devient une matrice 3×3 si l'orientation est intégrée dans les sorties.

8.5 Impact sur l'architecture logicielle

L'ajout du troisième degré de liberté impliquera de faire évoluer :

- `config.py` : ajout des paramètres de la troisième articulation ;
- `kinematics.py` : nouvelles équations de cinématique directe, inverse et jacobienne ;
- `simulator.py` : dessin d'un troisième segment ;
- `robot_controller.py` : animation et trajectoires à trois variables articulaires ;
- `gui.py` : nouveaux contrôles et nouvelle lecture d'état ;
- la couche GRBL : ajout d'un troisième axe abstrait, typiquement Z ou A.

8.6 Impact sur le prototype

Sur le plan matériel, cette évolution demandera :

- un moteur supplémentaire ;
- un driver supplémentaire ;
- une adaptation de la structure mécanique ;
- une révision du câblage et de l'alimentation ;
- une nouvelle phase de calibration.

8.7 Intérêt scientifique de cette perspective

L'intérêt de cette extension est double :

- elle transforme le projet en système plus proche d'un manipulateur utile pour la préhension ou l'orientation d'outil ;
- elle ouvre la voie à des études plus avancées sur la redondance, les contraintes de trajectoire, la commande et l'optimisation.

Cette perspective donne au projet une continuité scientifique forte et justifie pleinement son inscription dans un travail de recherche.

Chapitre 9

Conclusion générale

Le projet présenté constitue une base cohérente et solide pour l'étude d'un bras robotique plan à deux degrés de liberté. Sa valeur ne réside pas seulement dans l'existence d'une interface ou d'une animation, mais dans l'alignement entre :

- une théorie mathématique explicite ;
- une implémentation logicielle modulaire ;
- une stratégie de visualisation démonstrative ;
- une préparation concrète à l'intégration sur prototype via GRBL.

Les résultats actuels montrent que le système est capable :

- de calculer correctement les angles pour atteindre une cible ;
- de représenter de manière animée les mouvements du bras ;
- d'exécuter des trajectoires plus riches qu'un simple déplacement ponctuel ;
- de préparer une future validation expérimentale cohérente.

L'étape suivante, à savoir la construction du prototype puis l'ajout d'un troisième degré de liberté, n'apparaît donc pas comme une rupture, mais comme la continuité logique d'une architecture déjà pensée pour évoluer.

Annexe A

Extrait de code représentatif

Listing A.1 – Exemple simplifié de cinématique directe

```
def forward_kinematics(theta1, theta2, L1, L2):  
    x = L1 * np.cos(theta1) + L2 * np.cos(theta1 + theta2)  
    y = L1 * np.sin(theta1) + L2 * np.sin(theta1 + theta2)  
    return x, y
```

Listing A.2 – Exemple simplifié d'interpolation cartésienne

```
def compute_trajectory(start_pos, end_pos, num_points):  
    x_traj = np.linspace(start_pos[0], end_pos[0], num_points)  
    y_traj = np.linspace(start_pos[1], end_pos[1], num_points)  
    return list(zip(x_traj, y_traj))
```

Annexe B

Références bibliographiques

Bibliographie

- [1] M. W. Spong, S. Hutchinson, and M. Vidyasagar, *Robot Modeling and Control*, Wiley, 2006.
- [2] J. J. Craig, *Introduction to Robotics : Mechanics and Control*, Pearson, 3rd edition, 2005.
- [3] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics : Modelling, Planning and Control*, Springer, 2009.
- [4] GRBL Project, *GRBL Wiki*, <https://github.com/gnea/grbl/wiki>.
- [5] Pololu, *A4988 Stepper Motor Driver Carrier Documentation*, <https://www.pololu.com/product/1182>.